PRACTICAL : 01

Aim:

Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

1. Bubble Sort

Aim:

To implement Bubble Sort to sort a given array of elements in ascending order.

Algorithm:

1. Start.
2. Read the number of elements and the array.
3. Compare adjacent elements.
4. If the first element is greater than the second, swap them.
5. Repeat the process for all adjacent elements.
6. Continue the passes until the array is sorted.
7. Stop.

Code:

```
#include <iostream>
using namespace std;

class BubbleSort {
public:
   int size;
   int arr[100];

   void ArrayInput() {
      cout << "Enter the size of the array: ";
      cin >> size;
```

```
    cout << "Enter the array elements: ";
    for (int i = 0; i < size; i++) {
        cin >> arr[i];
    }
}

void printarray() {
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

void bubbleLogic() {
    for (int i = 0; i < size - 1; i++) {
        for (int j = 0; j < size - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
};

int main() {
    BubbleSort bs;
    bs.ArrayInput();

    cout << "Before sorting: ";
    bs.printarray();

    bs.bubbleLogic();

    cout << "After sorting: ";
```

```
    bs.printarray();

    return 0;
}
```

Output:

```
Enter the size of the array: 6
Enter the array elements: 34 56 54 45 23
4
Before sorting: 34 56 54 45 23 4
After sorting: 4 23 34 45 54 56
```

Time Complexity:

Best Case: O(n)
Average Case: O(n^2)
Worst Case: O(n^2)
Space Complexity: O(1)

2. Selection Sort

Aim:

To implement Selection Sort to sort a given array of elements in ascending order.

Algorithm:

1. Start.
2. Read the number of elements and the array.
3. Assume the first element of the unsorted portion is the minimum.
4. Compare it with the remaining elements.
5. Find the smallest element.
6. Swap it with the first element of the unsorted portion.

7. Repeat until the entire array is sorted.
8. Stop.

Code:

```
#include <iostream>
using namespace std;

class SelectionSort {
public:
   int size;
   int arr[100];

   void ArrayInput() {
      cout << "Enter the size of the array: ";
      cin >> size;

      cout << "Enter the array elements: ";
      for (int i = 0; i < size; i++) {
         cin >> arr[i];
      }
   }

   void printarray() {
      for (int i = 0; i < size; i++) {
         cout << arr[i] << " ";
      }
      cout << endl;
   }

   void selectionLogic() {
      for (int i = 0; i < size - 1; i++) {
         int min = i;

         for (int j = i + 1; j < size; j++) {
            if (arr[j] < arr[min]) {
               min = j;
```

```
            }
        }

        int temp = arr[i];
        arr[i] = arr[min];
        arr[min] = temp;
    }
  }
};

int main() {
  SelectionSort ss;
  ss.ArrayInput();

  cout << "Before sorting: ";
  ss.printarray();

  ss.selectionLogic();

  cout << "After sorting: ";
  ss.printarray();

  return 0;
}
```

Output:

```
Enter the size of the array: 5
Enter the array elements: 23 21 12 1 3 19
Before sorting: 23 21 12 1 3
After sorting: 1 3 12 21 23
```

Time Complexity:

Best Case: $O(n^2)$
Average Case: $O(n^2)$
Worst Case: $O(n^2)$
Space Complexity: O(1)

3. Insertion Sort

Aim:

To implement Insertion Sort to sort a given array of elements in ascending order.

Algorithm:

1. Start.
2. Read the number of elements and the array.
3. Consider the first element as the sorted portion.
4. Select the next element as the key.
5. Compare the key with the elements in the sorted portion.
6. Shift elements greater than the key one position to the right.
7. Insert the key at its correct position.
8. Repeat until all elements are sorted.
9. Stop.

Code:

```
#include <iostream>
using namespace std;

class InsertionSort {
public:
   int size;
   int arr[100];

   void ArrayInput() {
      cout << "Enter the size of the array: ";
      cin >> size;

      cout << "Enter the array elements: ";
      for (int i = 0; i < size; i++) {
         cin >> arr[i];
      }
   }
```

```
    void printarray() {
        for (int i = 0; i < size; i++) {
            cout << arr[i] << " ";
        }
        cout << endl;
    }

    void insertionLogic() {
        for (int i = 1; i < size; i++) {
            int key = arr[i];
            int j = i - 1;

            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j--;
            }

            arr[j + 1] = key;
        }
    }
};

int main() {
    InsertionSort is;
    is.ArrayInput();

    cout << "Before sorting: ";
    is.printarray();

    is.insertionLogic();

    cout << "After sorting: ";
    is.printarray();

    return 0;
}
```

Output:

```
Enter the size of the array: 4
Enter the array elements: 10 19 1 8
Before sorting: 10 19 1 8
After sorting: 1 8 10 19
```

Time Complexity:

Best Case: O(n)
Average Case: O(n^2)
Worst Case: O(n^2)
Space Complexity: O(1)

4. Merge Sort

Aim:

To implement Merge Sort to sort a given array of elements in ascending order.

Algorithm:

1. Start.
2. Read the number of elements and the array.
3. Divide the array into two halves.
4. Recursively divide each half until single elements remain.
5. Merge the divided subarrays in sorted order.
6. Continue merging until the complete array is sorted.
7. Stop.

Code:

```
#include <iostream>
using namespace std;

class MergeSort {
public:
   int size;
```

```
int arr[100];

void ArrayInput() {
   cout << "Enter the size of the array: ";
   cin >> size;

   cout << "Enter the array elements: ";
   for (int i = 0; i < size; i++) {
      cin >> arr[i];
   }
}

void printarray() {
   for (int i = 0; i < size; i++) {
      cout << arr[i] << " ";
   }
   cout << endl;
}

void merge(int low, int mid, int high) {
   int temp[100];
   int i = low;
   int j = mid + 1;
   int k = low;

   while (i <= mid && j <= high) {
      if (arr[i] <= arr[j]) {
         temp[k++] = arr[i++];
      } else {
         temp[k++] = arr[j++];
      }
   }

   while (i <= mid) {
      temp[k++] = arr[i++];
   }
```

```
        while (j <= high) {
            temp[k++] = arr[j++];
        }

        for (i = low; i <= high; i++) {
            arr[i] = temp[i];
        }
    }

    void mergeLogic(int low, int high) {
        if (low < high) {
            int mid = (low + high) / 2;

            mergeLogic(low, mid);
            mergeLogic(mid + 1, high);
            merge(low, mid, high);
        }
    }
};

int main() {
    MergeSort ms;
    ms.ArrayInput();

    cout << "Before sorting: ";
    ms.printarray();

    ms.mergeLogic(0, ms.size - 1);

    cout << "After sorting: ";
    ms.printarray();

    return 0;
}
```

Output:

```
Enter the size of the array: 7
Enter the array elements: 9 70 33 90 46 6 59
Before sorting: 9 70 33 90 46 6 59
After sorting: 6 9 33 46 59 70 90
```

Time Complexity:

Best Case: O(n log n)
Average Case: O(n log n)
Worst Case: O(n log n)
Space Complexity: O(n)

5. Quick Sort

Aim:

To implement Quick Sort to sort a given array of elements in ascending order.

Algorithm:

1. Start.
2. Read the number of elements and the array.
3. Select an element as the pivot.
4. Partition the array around the pivot.
5. Place smaller elements on the left of the pivot.
6. Place greater elements on the right of the pivot.
7. Recursively apply Quick Sort to the left and right partitions.
8. Stop when the partition contains zero or one element.

Code:

```
#include <iostream>
using namespace std;

class QuickSort {
public:
```

```
int size;
int arr[100];

void ArrayInput() {
   cout << "Enter the size of the array: ";
   cin >> size;

   cout << "Enter the array elements: ";
   for (int i = 0; i < size; i++) {
      cin >> arr[i];
   }
}

void printarray() {
   for (int i = 0; i < size; i++) {
      cout << arr[i] << " ";
   }
   cout << endl;
}

int partition(int low, int high) {
   int pivot = arr[high];
   int i = low - 1;

   for (int j = low; j < high; j++) {
      if (arr[j] <= pivot) {
         i++;

         int temp = arr[i];
         arr[i] = arr[j];
         arr[j] = temp;
      }
   }

   int temp = arr[i + 1];
   arr[i + 1] = arr[high];
   arr[high] = temp;
```

```
        return i + 1;
    }

    void quickLogic(int low, int high) {
        if (low < high) {
            int loc = partition(low, high);

            quickLogic(low, loc - 1);
            quickLogic(loc + 1, high);
        }
    }
};

int main() {
    QuickSort qs;
    qs.ArrayInput();

    cout << "Before sorting: ";
    qs.printarray();

    qs.quickLogic(0, qs.size - 1);

    cout << "After sorting: ";
    qs.printarray();

    return 0;
}
```

Output:

```
Enter the size of the array: 5
Enter the array elements:  99 78 34 2 1
Before sorting: 99 78 34 2 1
After sorting: 1 2 34 78 99
```

Time Complexity:

Best Case: O(n log n)
Average Case: O(n log n)
Worst Case: O(n^2)
Space Complexity: O(log n) average